



SISTEMAS INFORMÁTICOS

16/09/2026

Profesora:

Begoña Rodríguez Ferreras

begona.rodfer.1@educa.jcyl.es

RESULTADOS DE APRENDIZAJE

RA1. Evalúa **sistemas informáticos**, identificando sus componentes y características.

RA2. Instala **sistemas operativos** planificando el proceso e interpretando documentación técnica.

RA3. Gestiona la información del sistema identificando las estructuras de almacenamiento y aplicando medidas para asegurar la integridad de los datos.

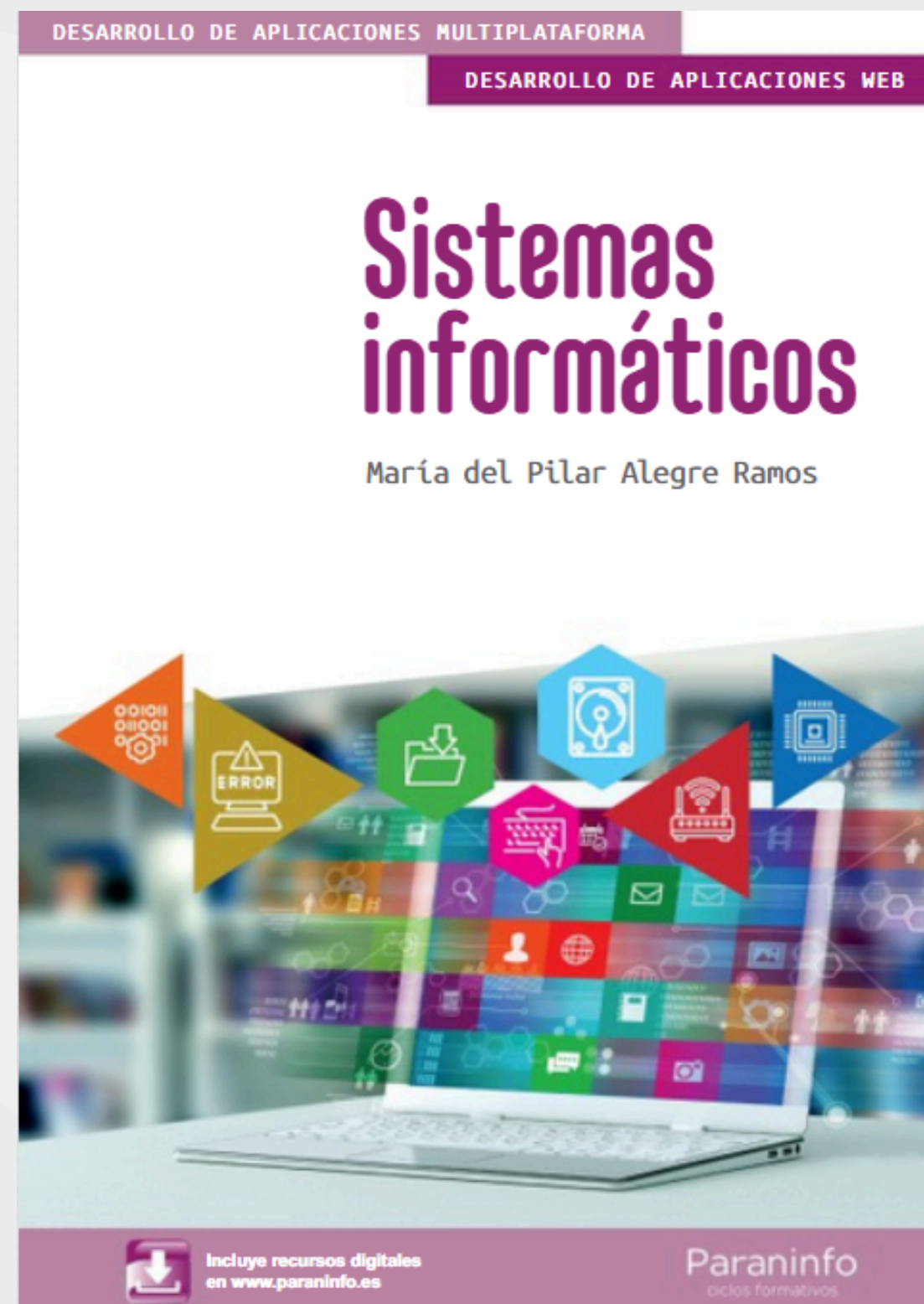
RA4. Gestiona sistemas operativos **utilizando comandos y herramientas gráficas** y evaluando las necesidades del sistema.

RA5. Interconecta **sistemas en red** configurando dispositivos y protocolos.

RA6. Opera **sistemas en red** gestionando sus recursos e identificando las restricciones de seguridad existentes.

RA7. Elabora **documentación** valorando y **utilizando aplicaciones informáticas de propósito general**.

LIBRO



RA1

- a) Se han reconocido los **componentes físicos** de un **sistema informático** y sus **mecanismos de interconexión**.
- b) Se han **clasificado los tipos de memorias**, señalando sus características e identificando sus prestaciones y la función que desarrollan en el conjunto del sistema.
- c) Se ha verificado el proceso de puesta en marcha de un equipo.
- d) Se han clasificado, instalado y configurado diferentes tipos de **dispositivos periféricos**.
- e) Se han identificado los **tipos de redes y sistemas de comunicación**.
- f) Se han identificado los **componentes de una red informática**.
- g) Se han interpretado **mapas físicos y lógicos** de una red informática.
- h) Se han reconocido las **normas de seguridad y prevención de riesgos laborales** en el uso de los sistemas informáticos.

UT1. EXPLOTACIÓN DE UN SISTEMA INFORMÁTICO: HARDWARE

- Componentes de un sistema informático. Elementos y conceptos fundamentales.
- La información y su representación. Sistemas de numeración y cambios de base. Operaciones lógicas.
- Unidades funcionales. Unidad central de proceso, Memoria y Buses.
- Periféricos. Adaptadores para la conexión de dispositivos.
- Normas de seguridad y prevención de riesgos laborales.

UT1. EXPLOTACIÓN DE UN SISTEMA INFORMÁTICO (II): REDES

- Tipos de redes.
- Componentes de una red informática.
- Topologías de red.
- Mapa físico y lógico de una red local.
- Medios de transmisión. Transmisión alámbrica e inalámbrica.
- Tipos de cableado. Cableado estructurado. Conectores.

COMPONENTES DE UN SISTEMA INFORMÁTICO

¿Qué es un **sistema informático**?

- Desde siempre, las personas han querido crear máquinas que les ayuden a resolver problemas.
- Al principio, cada máquina se diseñaba para una tarea concreta, por ejemplo, un reloj para medir el tiempo o una calculadora para hacer cuentas.
- Es un sistema que permite almacenar y procesar la información.

Pero entonces surgió la pregunta: ¿podemos construir una máquina que sirva para resolver diferentes problemas, no solo uno? La respuesta es sí, y esa máquina se llama sistema informático.

COMPONENTES DE UN SISTEMA INFORMÁTICO (II)

Está formado por tres **componentes**:

- **Hardware:** es la parte física y tangible. Son los componentes físicos del sistema informático: teclado, pantalla, procesador, disco duro...
- **Software:** aplicaciones que permiten operar con los diferentes componentes del sistema, como un juego, un procesador de texto o un navegador de Internet.
- **Personas:** persona o personas que utilizan el sistema informático

Hay autores que añaden:

- **Datos:** la información que el sistema procesa, por ejemplo, los documentos que escribes, las fotos que guardas o los vídeos que ves.

REPRESENTACIÓN Y COMUNICACIÓN DE LA INFORMACIÓN

- Si por algo se distingue al computador como máquina, es que su objeto de procesamiento es la **información**.
- La Informática es, como indica su propio nombre, el **procesamiento de información de forma automática**.
- El propósito de los ordenadores es, por tanto, el almacenamiento, transmisión y procesamiento de la información en general.
- Para que el procesamiento y almacenamiento que éste lleva a cabo es necesario escoger un formato adecuado de representación interna de la información. Igualmente, la transmisión de información requiere un sistema de representación que se adecúe a la tarea,

INFORMACIÓN

- El diccionario de la Real Academia Española de la Lengua define “información” como: *“acción y efecto de informar”, y como “comunicación o adquisición de conocimientos que permiten ampliar o precisar los que se poseen sobre una materia determinada”.*
- Podemos decir que un computador es una máquina que procesa datos (*“Información dispuesta de manera adecuada para su tratamiento por un ordenador”*), y los estructura y organiza (*información*), para permitir al ser humano obtener conocimiento (*nueva información*).

SISTEMAS DE NUMERACIÓN

¿Qué es un sistema de numeración?

- Conjunto de reglas, convenios y símbolos que permiten representar números
- Existen diferentes sistemas de numeración:
 - Sistema **decimal** (numeración en base 10): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Sistema **binario** (numeración en base 2): 0, 1
 - Sistema **octal** (numeración en base 8): 0, 1, 2, 3, 4, 5, 6, 7
 - Sistema **hexadecimal** (numeración en base 16): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

SISTEMAS DE NUMERACIÓN (II)

- Podemos definir también un sistema de numeración como un conjunto de dígitos y reglas que permiten **representar datos numéricos**.
- La principal regla es que **un mismo dígito tiene distinto valor según la posición que ocupe**.

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN

- Relaciona una cantidad expresada en cualquier sistema de numeración con la misma cantidad expresada en el sistema decimal; es decir, el valor decimal de una cantidad expresada en otro sistema de numeración que utiliza otra base.

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^{n-1} d_i b^i \\ N = d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

- i = posición respecto a la coma. Para los dígitos de la derecha la i es negativa empezando en $-k$; para los de la izquierda es positiva empezando en 0.
- k = número de dígitos a la derecha de la coma.
- n = número de dígitos a la izquierda de la coma.
- d = cada uno de los dígitos que componen el número.
- b = base del sistema de numeración (2,8,10 ó 16).

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN (II)

Si tenemos el número 555,55, el dígito 5 tiene distinto valor dependiendo de la posición que ocupa.

Cada posición tiene un peso asociado, siendo en este caso 5 las unidades, 50 las decenas y 500 las centenas. También serán 5 las décimas y 5 las centésimas.

El dígito más a la derecha tendrá peso -2, el siguiente -1, el siguiente 0, y así sucesivamente.

Podremos representar este número como las sumas de las potencias de la base 10 elevada al peso:

$$5 \cdot 10^{-2} + 5 \cdot 10^{-1} + 5 \cdot 10^0 + 5 \cdot 10^1 + 5 \cdot 10^2$$

EJERCICIO

Obtén el valor de los siguientes números siguiendo el teorema fundamental de la numeración:

327
74,23

TEOREMA FUNDAMENTAL DE LA NUMERACIÓN (III)

Las computadoras suelen efectuar las operaciones aritméticas utilizando una representación para los datos numéricos basada en el sistema de numeración base dos, que se denomina **binario**.

También se utilizan los sistemas de numeración, preferentemente el octal y hexadecimal, para obtener códigos intermedios. Un número expresado en uno de estos sistemas puede transformarse de forma directa a binario y viceversa.

Con ellos se simplifica la transcripción de número binarios y están más próximos al sistema que utilizamos usualmente (sistema digital), por los que a veces se utilizan como paso intermedio en las transformaciones de decimal a binario y viceversa.

EL SISTEMA DECIMAL

El sistema numérico, o BASE10, se utiliza a diario para los cálculos cotidianos, como la contabilidad doméstica, las unidades de medida, las horas, etc.

El sistema decimal utiliza diez dígitos o símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.

Dependiendo de la posición que ocupe un dígito dentro de una cifra, representará las unidades, decenas, centenas, millares, etc. Por esto, se dice que los sistemas de numeración son **posicionales**.

En BASE10 se utilizan las potencias de diez, por ejemplo, 23.605 en BASE10 significa lo siguiente: $(2 \cdot 10000) + (3 \cdot 1000) + (6 \cdot 100) + (0 \cdot 10) + (5 \cdot 1)$

EL SISTEMA BINARIO

El sistema de numeración binario utiliza solo dos dígitos (0 y 1) para representar cantidades, por lo que su base es 2. Cada dígito de un número representado por este sistema se denomina bit (*binary digit*).

Los bits tienen distinto valor dependiendo de la posición que ocupan; por eso este sistema también es posicional.

Estos valores vienen determinados por una potencia de base 2 que la vamos a llamar peso. Así, por ejemplo, el número binario 1011,01 expresado en decimal quedaría así:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 1 \cdot 8 + 1 \cdot 2 + 1 \cdot 1 + 1 \cdot 0,25 = 11,25$$

EL SISTEMA BINARIO (II)

Por tanto para pasar de binario a decimal será importante recordar los pesos asociados.

Pesos asociados								Número decimal
2^3	2^2	2^1	2^0	,	2^{-1}	2^{-2}	2^{-3}	
8	4	2	1	,	0,5	0,25	0,125	

Para trabajar con direccionamiento IP más adelante tendrán especial relevancia todos aquellos pesos hasta completar 1 byte (8 bits):

Binary Sample Place Value Chart							
128	64	32	16	8	4	2	1
↑	↑	↑	↑	↑	↑	↑	↑
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

CAMBIOS DE BASE

Al trabajar con diferentes bases se emplea la siguiente notación para indicar que un número está escrito en una base u otra:

- N_{10} para números en base decimal o base 10.
- N_2 para números binarios o en base 2.
- N_8 para números octales o en base 8.
- N_{16} para números hexadecimales o en base 16.

CAMBIOS DE BASE DE DECIMAL A BINARIO

Para cambiar de binario a decimal, el valor en decimal del número será la suma resultado de cada número multiplicado por la potencia de 2 de la posición que ocupa, tal y como se puede ver en el teorema fundamental de la numeración:

$$N_{(10)} = \sum_{i=1}^{k-1} d_i \cdot 2^i$$

Calcula el valor en decimal de los siguientes números en binario.

$100_{(2)}$
 $1010_{(2)}$
 $1101_{(2)}$
 $10011_{(2)}$

EJERCICIO

Calcula el valor en decimal de los siguientes números en binario:

100

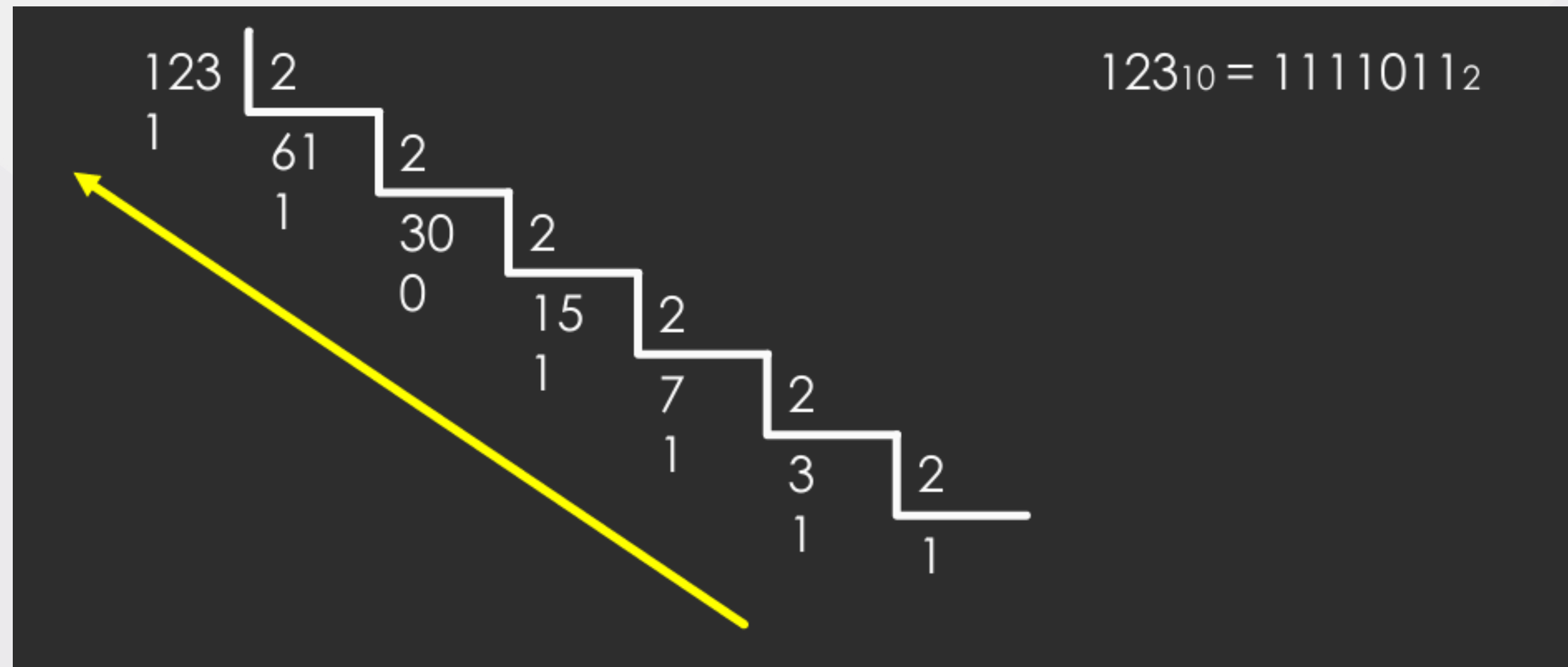
1010

1101

10011

CAMBIO DE BASE DECIMAL A BINARIA

Conversión de 123 a binario:



EL SISTEMA OCTAL

Los primeros sistemas informáticos utilizaban solo el sistema binario para interpretar y transformar los datos, con lo que las labores de programación eran bastante tediosas; se recurrió entonces al uso de sistemas intermedios que permitían una fácil traducción hacia y desde el sistema binario. Estos sistemas son el octal y el hexadecimal.

El sistema octal tiene como base de numeración 8, es decir, utiliza ocho símbolos para representar las cantidades. Estos símbolos son 0, 1, 2, 3, 4, 5, 6, 7. Este sistema también es posicional; o sea, que un mismo dígito tiene distinto valor según la posición que ocupe.

EL SISTEMA HEXADECIMAL

El sistema hexadecimal tiene como base de numeración 16, es decir, utiliza dieciséis símbolos para representar las cantidades. Este sistema también es posicional. A los símbolos A, B, C, D, E y F se les asignan los siguientes valores:

Símbolo	Valor asignado
A	10
B	11
C	12
D	13
E	14
F	15

EL SISTEMA HEXADECIMAL (II)

La numeración en hexadecimal tiene la utilidad de que:

- 4 números binarios se pueden representar directamente con un número hexadecimal
- un byte se representa con dos números hexadecimales

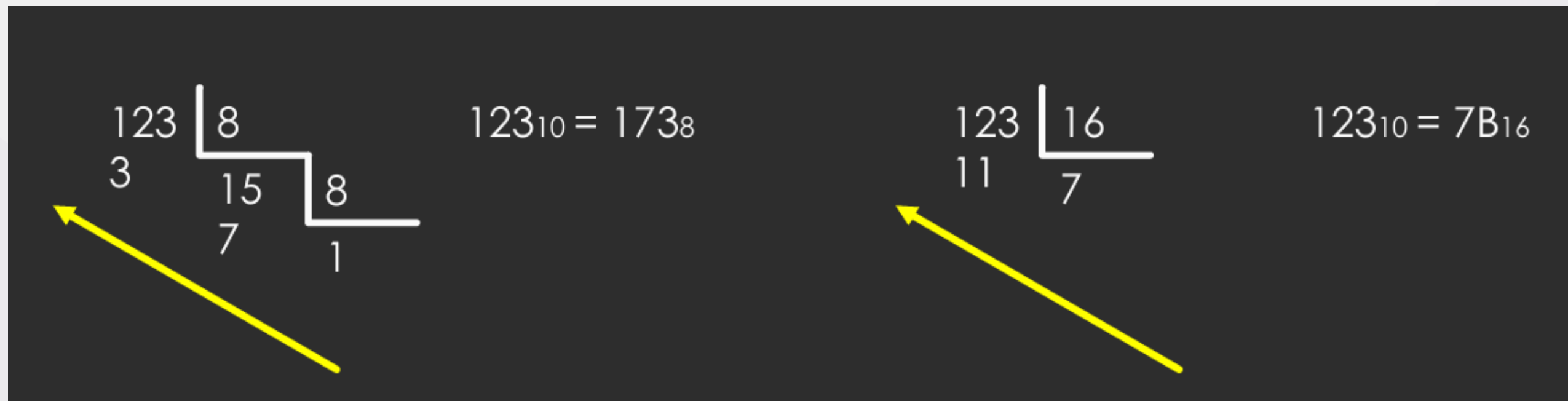
Se utiliza para representar las direcciones de memoria, las direcciones MAC de las tarjetas de red, las direcciones IPv6...

El motivo de que su uso esté tan extendido es que para los seres humanos es más fácil y simple leer o escribir valores hexadecimales que números binarios.

CAMBIO DE BASE

DECIMAL A OCTAL Y HEXADECIMAL

Conversión de 123 a octal y hexadecimal:



EQUIVALENCIAS ENTRE SISTEMAS NÚMERICOS

Decimal	Binario	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

OPERACIONES LÓGICAS

En el sistema binario se pueden realizar una serie de operaciones lógicas que se llevan a cabo siguiendo el **álgebra booleana o de Boole**, llamada así en honor al matemático inglés George Boole, que fue el que definió las operaciones lógicas. Internamente un ordenador trabaja con este tipo de lógica.

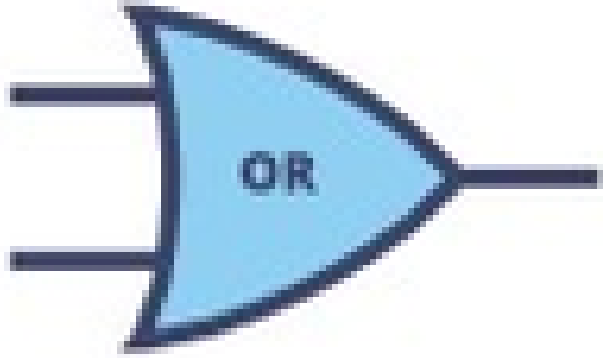
Las operaciones lógicas básicas son:

- **OR:** el resultado de esta operación lógica será 1 cuando alguno de los valores sea 1.
- **AND:** el resultado de esta operación lógica será 1 solo cuando los dos valores sean 1.
- **NOT:** el resultado de esta operación lógica será negar el valor que recibe.

FUNCIÓN LÓGICA BÁSICA OR

El resultado de esta operación lógica será 1 cuando alguno de los valores sea 1.

A	B	OR
0	0	0
0	1	1
1	0	1
1	1	1

A diagram of a blue OR gate with two inputs on the left and one output on the right. The word "OR" is written inside the gate.

FUNCIÓN LÓGICA BÁSICA AND

El resultado de esta operación lógica será 1 solo cuando los dos valores sean 1.

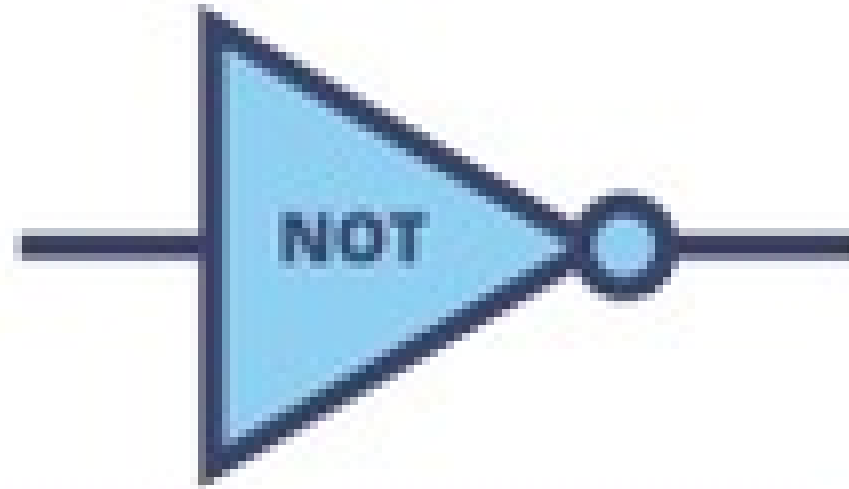
A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

A blue AND gate symbol with two input lines on the left and one output line on the right. The word "AND" is written inside the gate.

FUNCIÓN LÓGICA BÁSICA NOT

El resultado de esta operación lógica será negar el valor que recibe.

A	NOT
0	1
1	0



EJERCICIO

Para los siguientes pares de números en binario, realiza las operaciones lógicas OR y AND:

01111100
10001001

OPERACIONES LÓGICAS DERIVADAS

Las operaciones lógicas derivadas son:

- **NOR:** el resultado de la negación de OR. $A \text{ NOR } B = \text{NOT } (A \text{ OR } B)$.
- **NAND:** es la negación de AND. $A \text{ NAND } B = \text{NOT } (A \text{ AND } B)$.
- **XOR:** el resultado de las siguientes operaciones:
 $A \text{ XOR } B = (\text{NOT } (A) \text{ AND } B) \text{ OR } (A \text{ AND } (\text{NOT } B))$.

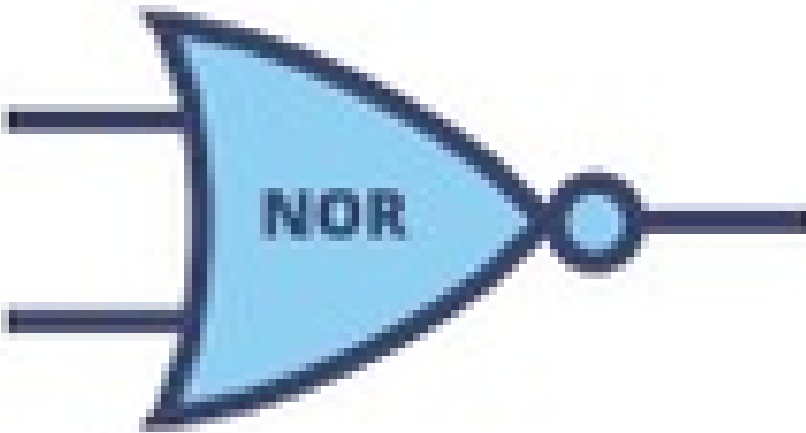
El resultado de esta operación lógica será 0 cuando ambos valores sean iguales y 1 cuando sean diferentes.

FUNCIÓN NOR

El resultado de la negación de OR.

$A \text{ NOR } B = \text{NOT } (A \text{ OR } B)$.

A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0

A blue NOR gate symbol with two inputs on the left and one output on the right. The output line ends in a small circle, representing inversion. The word "NOR" is written inside the gate.

FUNCIÓN NAND

Es la negación de AND.

$A \text{ NAND } B = \text{NOT } (A \text{ AND } B)$.

A	B	NAND
0	0	1
0	1	1
1	0	1
1	1	0



FUNCIÓN XOR

El resultado de las siguientes operaciones:

$$A \text{ XOR } B = (\text{NOT } (A) \text{ AND } B) \text{ OR } (A \text{ AND } (\text{NOT } B)).$$

El resultado de esta operación lógica será 0 cuando ambos valores sean iguales y 1 cuando sean diferentes.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0



OPERACIONES ARITMÉTICAS: SUMA

Para realizar la operación aritmética suma sería necesario aplicar la siguiente tabla:

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

El valor que aparece en la columna Acarreo es la cantidad que se arrastra para la operación de los siguientes dígitos a la izquierda.

OPERACIONES ARITMÉTICAS: SUMA (II)

Para sumar los números en binario se puede colocar el acarreo sobre la columna donde se tiene que aplicar.

En el siguiente ejemplo se pueden observar dos sumas de números binarios (junto a cada número y en el resultado se expresa en color azul su valor en decimal):

$\begin{array}{r} \textcolor{brown}{1111} \\ 00100110_{(2)} \\ + 10111101_{(2)} \\ \hline 11100011_{(2)} \end{array}$	$\begin{array}{r} 38_{(10)} \\ 189_{(10)} \\ 227_{(10)} \end{array}$	$\begin{array}{r} \textcolor{brown}{11111} \\ 01101100_{(2)} \\ + 10111001_{(2)} \\ \hline 100100101_{(2)} \end{array}$	$\begin{array}{r} 108_{(10)} \\ 185_{(10)} \\ 293_{(10)} \end{array}$
---	--	---	---

EJERCICIO

Suma los siguientes números en binario:

```
00110111
+ 10110101
          
```

```
10101100
+ 00101001
          
```


OPERACIONES ARITMÉTICAS: RESTA

Para restar en binario se usan los complementos que utiliza el ordenador internamente para almacenar números negativos.

Realizar una resta consistiría en sumar el primer número con el complemento del segundo.

Para realizar los complementos se utilizan números de igual longitud y de longitud fija:

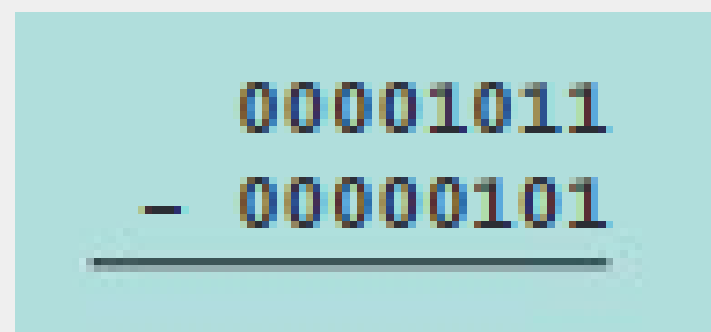
- **Complemento a 1:** sería el resultado de realizar la operación NOT sobre el número
- **Complemento a 2:** sería el resultado de realizar el <<complemento a 1>> sobre el número y sumarle 1 al resultado (solo si el número es negativo).

EJERCICIOS COMPLEMENTOS Y RESTA

- Calcula el complemento a 1 y el complemento a 2 del siguiente número utilizando 8 dígitos:

00001010

- Resta los números binarios utilizando el complemento a 2. Comprueba que los resultados son correctos convirtiendo cada número a decimal.



A light blue rectangular box containing a binary subtraction problem. The first number is 00001011, and the second number is 00000101, preceded by a minus sign. A horizontal line is drawn under the second number. The numbers are displayed in a monospaced, pixelated font.

$$\begin{array}{r} 00001011 \\ - 00000101 \\ \hline \end{array}$$

ALMACENAMIENTO DE LA INFORMACIÓN

Internamente, el ordenador utiliza el **sistema binario** para almacenar en la memoria tanto los **números enteros** y **reales** como los **caracteres**. Existen varios métodos para almacenarlos.

Para **almacenar números enteros** puede utilizar:

- La representación de entero sin signo (almacena un número entero positivo).
- Entero con signo y magnitud (donde el bit más a la izquierda representa el signo, de tal forma que un 0 es positivo y un 1 es negativo).
- Complemento a 1.
- **Complemento a 2.** En realidad es el estándar actual porque no tiene problemas de doble representación de ceros. Así además, la suma y resta funcionan con la misma circuitería, sin necesidad de distinguir el signo

ALMACENAMIENTO DE LA INFORMACIÓN

Almacenamiento de números enteros (II):

Por ejemplo, la representación de un número entero con signo utilizando 4 bytes sería para un número positivo y uno negativo la siguiente:

byte 1	byte 2	byte 3	byte 4	
00000000	00000000	00111101	00111110	(almacena el número $15.678_{(10)}$)
10000001	10001011	00100110	01100011	(almacena el número $-25.896.547_{(10)}$)

Utilizando 4 bytes, en **entero con signo**, los números mayor o menor que se pueden almacenar son $2^{31} - 1$ y -2^{31} . En entero sin signo el mayor sería $2^{32} - 1$ y el menor 0.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE

- El almacenamiento de los números en coma flotante se realiza siguiendo el **estándar IEEE 754**.
- Al igual que en la notación científica, se almacenan de la siguiente forma: signo (1 bit), mantisa y el exponente, cuyas longitudes variarán dependiendo de la precisión del número.
- La base será 2 al tratarse del sistema binario, por lo que no será necesario almacenarla.
- El exponente o característica e , se calcula, una vez fijado, con el sesgo en correspondencia con el ancho en bits de este campo. Esta representación sesgada evita tener que almacenar otro bit de signo para el exponente: $e = E + S$.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (II)

- Los números se pueden almacenar en varios bytes, de manera que puede hablarse de precisión simple (4 bytes), doble (8 bytes) y otros formatos.
- En la **precisión simple** (4 bytes): se almacenaría el número utilizando para el signo 1 bit, para el exponente 8 bits (Sesgo = 127) y para la mantisa 23 bits.
- En la **precisión doble** (8 bytes), el signo sería 1 bit, 11 bits el exponente (Sesgo = 1023) y 52 bits la mantisa.

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (III)

Por ejemplo, en precisión simple, se almacenaría el número $48,50_{(10)}$ de la siguiente forma:

- Convertimos el número a binario $110000,1_{(2)}$.
- En binario la notación científica exige que quede como $1, \text{mantisa} \times 2^{\text{exponente}}$
- Movemos la coma hasta dejar un único 1 a la izquierda: $110000,1_{(2)} = 1,100001 \times 2^5$
- El exponente sería $5_{(10)}$ a este número le tenemos que sumar el sesgo con lo que $e=5+127=132$ y en binario $10000100_{(2)}$. Para el exponente se almacena también el signo, que es el primer bit y es 0 si es negativo y 1 si es positivo.
- La mantisa tiene un tamaño fijo, y si tu número normalizado no ocupa todos los bits, el estándar exige completar el resto con ceros.

signo	exponente	mantisa
0	10000100	100001000000000000000000

ALMACENAMIENTO DE NÚMEROS EN COMA FLOTANTE (IV)

- La mantisa tiene un tamaño fijo, y si tu número normalizado no ocupa todos los bits, el estándar exige completar el resto con ceros.
- En la mantisa, en este formato, se utiliza el bit implícito, que significa que se va a correr la coma hasta el primer 1, pero se deja a la derecha del primer bit a uno. Dicho bit implícito no será representado, porque se sabe que es un 1, con lo que se aumenta en un bit más la precisión de la mantisa.
- El número del ejemplo se almacenaría en 4 bytes de la siguiente forma:

signo	exponente	mantisa
0	10000100	100001000000000000000000

byte 1	byte 2	byte 3	byte 4
01000010	01000010	00000000	00000000

ALMACENAMIENTO DE CARACTERES

Para los caracteres se utilizan los denominados códigos de entrada y salida, o E/S:

- **código ASCII:** 7 bits; los primeros 32 caracteres son de control y los demás representan números, letras y caracteres especiales.
- **ASCII extendido:** 8 bits, que amplió el anterior para añadir caracteres del idioma inglés.
- **ISO-8859-1 o Latin-1:** 8 bits, en el que los 128 primeros caracteres coinciden con la codificación ASCII.
- **ISO-8859-15 o Latin 9:** es el anterior al que se le añadieron algunos caracteres especiales, como el símbolo del €.
- **Unicode:** estándar muy utilizado del que existen tres tipos de codificaciones: UTF-8, UTF-16 y UTF-32
- **y otros...**
- En la actualidad, casi exclusivamente se usa el Unicode 15.1 y UTF-8 como estándar en la Web.

ELEMENTOS FUNCIONALES DE UN ORDENADOR

Un computador u ordenador es una máquina electrónica capaz de procesar datos, aceptarlos desde una unidad de entrada, manipularlos mediante operaciones aritméticas y lógicas, y presentar los resultados a través de una unidad de salida. Su funcionamiento se basa en un programa almacenado en memoria que controla de forma automática todas las operaciones.

El ordenador digital utiliza un esquema interno de funcionamiento basado en representar la información con un conjunto de valores discretos (normalmente dos valores distintos). Las operaciones se realizan aplicando una serie de instrucciones sobre los dígitos que representan la información.

EVOLUCIÓN HISTÓRICA: COMPUTADORAS NO DIGITALES

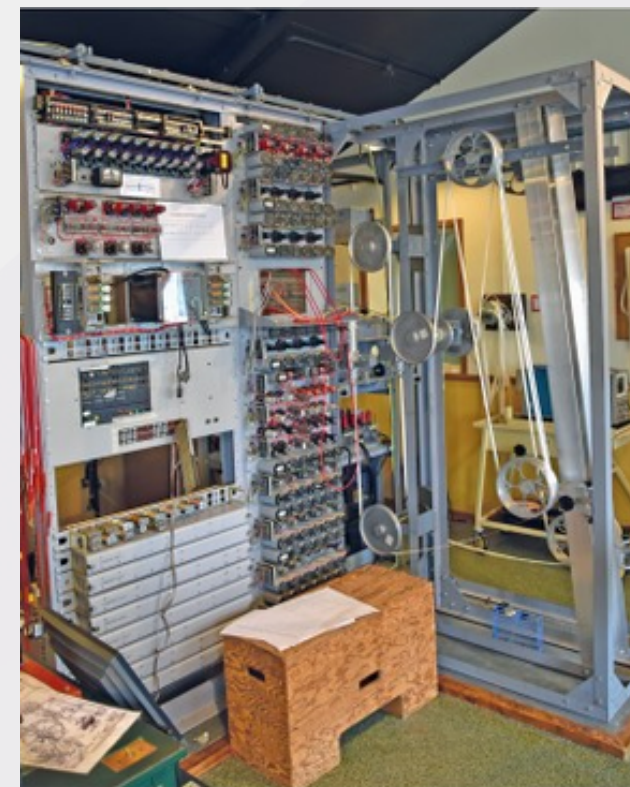
- El **ábaco**, fue una de las primeras herramientas para realizar cálculos aritméticos de forma rápida. Aunque no era un ordenador en el sentido moderno, marcó el inicio del uso de dispositivos para el procesamiento de datos.
- En el siglo XVII, **Blaise Pascal** creó la Pascalina, una calculadora mecánica capaz de sumar y restar automáticamente. Más tarde, en el siglo XIX, **Charles Babbage** diseñó la Máquina Analítica, considerada el primer concepto de ordenador programable, aunque nunca llegó a completarse.
- Paralelamente, se establecieron fundamentos esenciales para la informática actual: el **sistema binario** propuesto por **Leibniz** (siglo XVII), el **álgebra de Boole** (George Boole, siglo XIX), y la **máquina de Turing**, descrita por Alan Turing en 1936 como modelo teórico de computación...

EVOLUCIÓN HISTÓRICA: COMPUTADORAS DIGITALES

- El uso de la **electricidad** permitió construir ordenadores más rápidos, potentes y fiables que los modelos mecánicos anteriores.
- El primero de este tipo fue el **Z1**, diseñado por el ingeniero alemán Konrad Zuse en 1938. Poco después, en 1939, John Atanasoff y Clifford Berry crearon en EE. UU. el **ABC (Atanasoff-Berry Computer)**.
- Ambos fueron pioneros al usar interruptores eléctricos para representar información: un interruptor apagado indicaba un 0, y uno encendido, un 1.

PRIMERA GENERACIÓN (1946-1955)

- Ordenadores basados en **válvulas electrónicas de vacío**.
- Tenían un tamaño muy grande y su mantenimiento resultaba muy complicado.
- Se destinaban a aplicaciones para el campo científico y militar (Segunda Guerra Mundial: <https://www.youtube.com/watch?v=MCljmxr4Ujs>).
- Utilizaban como lenguaje de programación el **lenguaje máquina**, no disponían de sistema operativo y empleaban la **tarjeta perforada** para almacenar la información. Ejemplo de estos son el ENIAC y el UNIVAC1.



SEGUNDA GENERACIÓN (1955-1964)

- Se sustituye la válvula de vacío por el **transistor**, lo que influye en que los ordenadores puedan ser más pequeños, generen menos calor y sean más económicos.
- Comenzaron a emplearse lenguajes de alto nivel: Cobol, Algol y Fortran.
- También, memorias de núcleos de ferrita, cintas y tambores magnéticos para almacenar la información. Ejemplo de ordenador (IBM 1401).
- Ejemplo de uso en escenas de cine (Figuras Ocultas):
<https://www.youtube.com/watch?v=1C5lc2GlisQ>

TERCERA GENERACIÓN (1964-1974)

- Ordenadores basados en **circuitos integrados** (en un solo chip se integran varios transistores). Esto supuso la minimización de los ordenadores, así como el aumento notable de la velocidad.
- En esta época evolucionó el software de forma considerable, sobre todo en que se desarrollan los **sistemas operativos**, en los que se incluyó la multiprogramación y el tiempo real.
- También evolucionaron apreciablemente las unidades de almacenamiento, y aparecieron los discos magnéticos.
- Comenzaron a utilizarse memorias de semiconductores...
- Ejemplo de ordenador (IBM 370).



Figura 1.9. Comparación del tamaño de un chip, un transistor y un tubo de vacío.

CUARTA GENERACIÓN (1974-1983)

- La componen los ordenadores que presentan toda la CPU en un solo circuito integrado: los **microprocesadores**.
- El primer micro (el 4004) lo desarrolló Intel en 1971.
- Continúa la reducción del tamaño y del coste de los ordenadores
- Empieza a utilizarse el ordenador para uso personal o doméstico.
- Surgen los sistemas operativos CP/M y Unix, precursor del actual Linux.
- En esta generación hacen su aparición las **aplicaciones de uso general y de gestión**.

ESCALAS DE INTEGRACIÓN

Dependiendo del número de transistores en un chip se habla de diferentes escalas de integración:

Nivel de integración	Componentes integrados en un chip
SSI (<i>Small Scale Integration</i>)	10-100
MSI (<i>Medium Scale Integration</i>)	101-1.000
LSI (<i>Large Scale Integration</i>)	1.001-10.000
VLSI (<i>Very Large Scale Integration</i>)	10.001-100.000
ULSI (<i>Ultra Large Scale Integration</i>)	100.001-1.000.000
GLSI (<i>Giga Large Scale Integration</i>)	1.000.001 o más

QUINTA GENERACIÓN (DESDE 1983)

- En 1983, Japón lanzó el llamado Programa de la Quinta Generación de Computadoras, con los objetivos explícitos de producir máquinas capaces de comunicarse en un **lenguaje más cotidiano** y no a través de códigos o lenguajes de control especializados.
- Hay autores que ponen el comienzo en 1981 con la comercialización del PC (Personal Computer) por parte de IBM.
- Surgen los sistemas operativos MS-DOS, Windows, Linux y Mac OS.
- Se desarrolla la programación orientada a objetos (POO) y surgen lenguajes como Java, C++, C# y Delphi, entre otros.

QUINTA GENERACIÓN (DESDE 1983) (II)

- Esta generación sigue hasta nuestros días
- Parece que la próxima generación será la que venga marcada por el desarrollo y la generalización de los ordenadores cuánticos.

¿PREGUNTAS?